UNIVERSIDAD AMERICANA

Facultad de Ingeniería y Arquitectura (**FIA**)
Asignatura: Álgebra Lineal (**MTM0120**)

ACTIVIDADES DE PROGRAMACIÓN EN PYTHON
PROGRAMA 6

Factorización LU $(A = LU)$ y solución de sistemas $Ax = b$
Calculadora de Álgebra Lineal

Modalidad

Grupal (Máximo **4** estudiantes) con defensa/exposición individual.

Segundo Corte Evaluativo
Valor: 10 puntos

1. ¿De qué trata esta actividad?

En esta actividad van a desarrollar un módulo en Python que obtenga la Factorización LU de una matriz A $(A = LU)$ mediante operaciones elementales de reemplazo de filas, verifique el resultado $L \cdot U = A$ y utilice la descomposición para resolver el sistema $Ax = b$ en dos etapas sustitutivas:

1. Sustitución hacia adelante: Resolver $Ly = b$ para hallar el vector intermedio y.
2. Sustitución hacia atrás: Resolver $Ux = y$ para determinar la solución final x.

Recuerden que L es una matriz triangular inferior con unos en la diagonal principal ($m\ x\ m$) que registra ordenadamente los multiplicadores de eliminación, y U es la forma escalonada de A ($m\ x\ n$).

Este módulo se integra formalmente a la Calculadora de Álgebra Lineal como $modulos/modulo_lu.py$. Asimismo, deberán aplicar las mejoras obligatorias en los verificadores de propiedades y encadenamiento de datos derivadas de las revisiones de las Tareas 4 y 5.

2. Reglas del Juego (Técnicas)

1. **Cero librerías externas**: Queda estrictamente prohibido el uso de NumPy, SciPy, o cualquier función de álgebra de librerías externas o del módulo math. Todo el código debe basarse en sintaxis nativa de Python (listas anidadas, condicionales if/else, bucles for/while y funciones personalizadas). Se permite la importación de fractions.Fraction de la biblioteca estándar para representación exacta de multiplicadores y fracciones.

2. **Reutilización de código**: Utilicen las funciones previamente programadas en la Calculadora (producto de matrices, transpuesta, eliminación Gaussiana, comprobación de igualdad) importándolas directamente desde la carpeta modulos/; no las reescriban.
3. **Funciones puras que devuelven resultados**: Los módulos de cálculo NO deben contener input() ni print(). Deben recibir estructuras de datos (listas anidadas) y retornar los resultados (return), permitiendo su encadenamiento directo.
4. **Código documentado con criterio**: Aplicación rigurosa de docstrings por módulo y función, restringiendo los comentarios de línea exclusivamente a explicar la toma de decisiones matemáticas (ver Sección 6).

3. Requerimientos del programa

Entrada e Interfaz:

- Incorporar la opción "Factorización LU y solución $Ax = b$" en el menú principal de la Calculadora (main.py).
- Presentar el Logotipo ASCII representativo del módulo y la opción $0. Ver\ Teoremas\ Clave\ del\ Módulo$.
- Solicitar las dimensiones $m\ x\ n$ de la matriz A y sus componentes numéricas, seguidas del vector de términos independientes b de m componentes.
- Validar que los datos sean números válidos; ante un error, el sistema debe notificar y solicitar la reentrada sin colapsar.

Procesamiento:

1. **Obtención de U y registro de L:**
   - Reducir A a su forma escalonada U aplicando únicamente operaciones de reemplazo de fila ($F_i \leftarrow F_i - l \cdot F_j$).
   - Calcular y almacenar cada multiplicador $l = \frac{elemento\ a\ eliminar}{pivote}$ en la posición correspondiente (i, j) de la matriz L.
   - La matriz L debe estructurarse como triangular inferior de orden $m\ x\ m$ con unos (1) en la diagonal principal.
   - **Manejo de Permutaciones**: Si un pivote en la diagonal resulta ser cero y requiere un intercambio de filas para continuar, el programa debe detener el proceso e informar: "La matriz no admite Factorización LU sin permutación (requiere pivoteo parcial $PA = LU$)".

2. **Verificación de la Factorización:**
   - Multiplicar $L \cdot U$ invocando la función de producto matricial de la Calculadora y evaluar si $A \cdot x == b$.
3. **Resolución del sistema $Ax = b$ (Para matrices cuadradas $n\ x\ n$):**

- Resolver $Ly = b$ mediante reducción sobre $[L|b]$ para obtener y.
- Resolver $Ux = y$ mediante reducción sobre $[U|y]$ para obtener x.
- Comprobar la exactitud evaluando $A \cdot x == b$.

Salida esperada:

- Despliegue con formato visual limpio de las matrices L y U.
- Mensaje explícito de verificación: "La factorización es correcta ($L \cdot U == A$)".
- Despliegue del vector intermedio y y del vector solución x.
- Verificación final: "Comprobación exitosa: $A \cdot x == b$".

4. Mejoras obligatorias a la Calculadora (observaciones de las Tareas 4 y 5)

Con base en la evaluación de los entregables anteriores, cada grupo debe incorporar las siguientes correcciones al proyecto global:

a) Cobertura Completa de Propiedades

El verificador de propiedades ($modulos/modulo_matrices.py$ / $teoremas/$) debe calcular ambos miembros de cada igualdad utilizando las funciones internas del sistema y desplegar el diagnóstico "$Se\ cumple$" o "$No\ se\ cumple$" para el siguiente conjunto de teoremas:

Categoría	Propiedades que el verificador debe evaluar (ambos miembros)
Operaciones y transposición	$(A^T)^T = A$ $(A+B)^T = A^T + B^T$ $(kA)^T = kA^T$ $(AB)^T = B^T A^T$ $A(B + C) = AB + AC$ $(AB)C = A(BC)$
Matriz inversa	$(A^{-1})^{-1} = A$ $(AB)^{-1} = B^{-1}A^{-1}$ $(A^T)^{-1} = (A^{-1})^T$ $A \cdot A^{-1} = I$

Categoría	Propiedades que el verificador debe evaluar (ambos miembros)
Determinantes	$det(A^T) = det(A)$ $det(AB) = det(A) \cdot det(B)$ $det(A^{-1}) = 1/det(A)$ $det(kA) = k^n\, det(A)\ (para\ A_{n\,x\,n})$ Efecto de operaciones de fila: permutación cambia signo, reemplazo conserva det, multiplicación de fila por k escala el det por k. Determinante de matriz triangular = producto de elementos diagonales.

b) Encadenamiento de resultados mediante listas anidadas

Prohibido solicitar al usuario la reentrada de datos intermedios. Las funciones de cálculo deben retornar listas anidadas que se almacenen en variables y se transfieran directamente como argumentos de otras funciones.

Ejemplo de implementación limpia para $(AB)^T = B^T A^T$:

```
# AB se calcula una vez y se reutiliza, no se vuelve a escribir
AB = producto(A, B)
izquierda = transpuesta(AB)
derecha = producto(transpuesta(B), transpuesta(A))
print("(AB)^T = B^T A^T:", son_iguales(izquierda, derecha))
```

5. Formato de entrega

Subir a la plataforma Moodle un archivo comprimido .zip nombrado estrictamente:

$Programa\ 6_Grupo\ x_Apellido1_Apellido2.zip$

Contenido del archivo:

1. **Código Fuente del Proyecto**: La carpeta completa del proyecto

   $Calculadora_Algebra_Lineal/$ conteniendo $main.py$, la carpeta $modulos/$ (con $modulo_lu.py$ y las mejoras aplicadas) y $teoremas/$.

2. **Reporte en PDF**: Nombrado $Informe_Programa\ 6_Grupo\ x.pdf$, estructurado de la siguiente manera:
   - Portada Oficial: Universidad Americana, FIA, Álgebra Lineal, Docente, Integrantes, Carrera y Grupo.

- Explicación Algorítmica (1 a 2 páginas): Descripción analítica del procedimiento para extraer los multiplicadores de L, la reducción a U, el proceso de doble sustitución $(Ly = b, Ux = y)$ y la arquitectura del encadenamiento de funciones.

6. Guía de documentación del código: comentar con criterio

La documentación debe aportar valor técnico sin generar ruido visual. Se deben acatar las siguientes normas:

- **Docstring de Módulo**: Ubicada al inicio de cada archivo $.py$, indicando el propósito técnico, unidad del curso y autores.
- **Docstring por función (1 a 3 líneas)**: Descripción sucinta de qué calcula, qué parámetros recibe (entradas) y qué retorna (salida).
- **Comentarios de Línea para la Justificación** $(porque)$: Reservados exclusivamente para decisiones matemáticas complejas (ej. "Se registra el multiplicador de eliminación en L antes de alterar la fila en U").
- **Nombres Autoexplicativos**: Utilizar identificadores claros como $matriz_L, matriz_U, multiplicador_l, vector_y$ en lugar de variables genéricas $(a, temp, aux1)$.
- **Cero Código Muerto**: No entregar líneas de código inactivas o comentadas.

Ejemplo de documentación esperada:

```
def calcular_multiplicador(U, fila, fila_pivote, col):
    """Devuelve l tal que Fi <- Fi - l*Fj anula U[fila][col].

    U: lista anidada (m x n). Requiere U[fila_pivote][col] != 0.
    """
    # El multiplicador se guarda aparte porque L lo necesita despues
    return U[fila][col] / U[fila_pivote][col]
```